

# Serverless Spatial

## Vector Tiles using DuckDB, GeoParquet & Azure Storage+Functions

*Jesse Amundsen*
*2026*

# Introduction

- From Cincinnati, OH and Kentucky
- Approaching two decades in telecom working on geospatial problems
- Led the technical development of the Broadband Serviceable Location Fabric (FCC, NTIA)



**Jesse Amundsen**
Principal Developer
CostQuest Associates

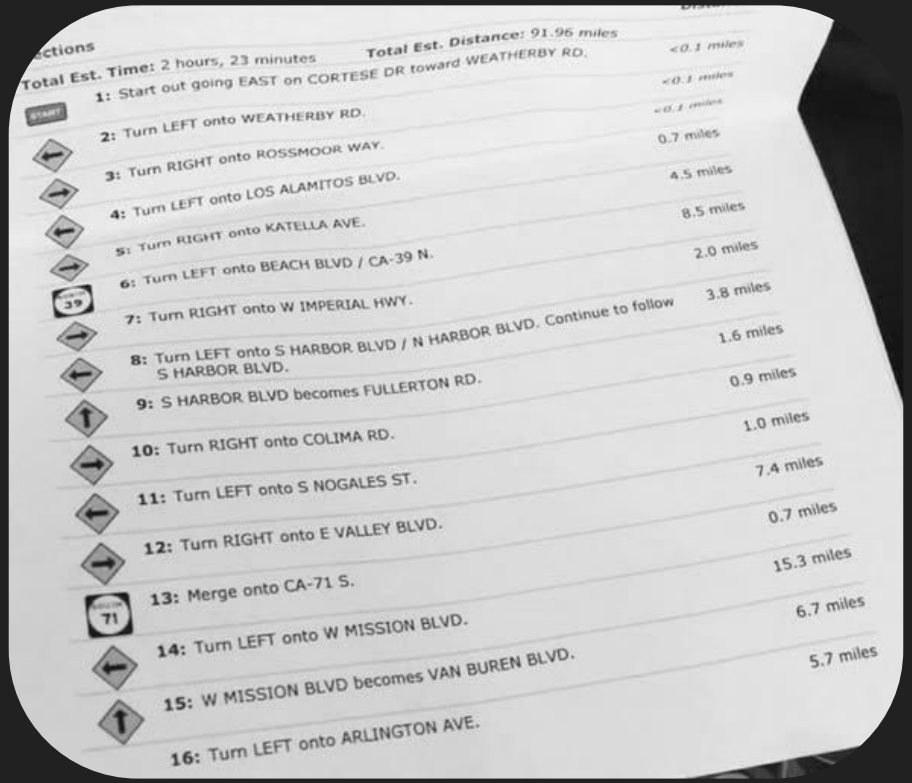
# Moving fast

- AWS acquires DuckLabs
  - https://aws.amazon.com/blogs/big-data/aws-and-ducklabs-building-the-future-of-analytics-together/
- DuckDB 2.0.0 (released 2026-10-21)
  - Continues the tradition of global performance improvements
  - Push-down / pruning for bounding box filtering with spatial predicates
  - row_group_size_bytes / order_by support
  - References
    - https://duckdb.org/2026/10/21/announcing-duckdb-200
    - https://github.com/duckdb/duckdb/issues/27247
- DuckDB backend support
  - In the works for martin, tileserver-rs, etc.



# Web Mapping

- We have come a long way from printing out MapQuest directions
- The expectations today are that our maps more closely resemble a GIS application than a piece of paper
- The content has become increasingly dynamic, allowing deeper interaction from users




*Source: 1980sGamer on Facebook*

# Vector Tiles: Quick History

- We used to be stuck with less customizable raster tiles
- Mapbox Vector Tiles (MVT) took over in the mid 2010's
  - Customizable on the client
  - Rendered cleanly at varying zoom levels
  - Useful for *features*
  - Supports feature attributes
- Delivered using {z}/{x}/{y}

# Vector Tiles: Static

- Tippecanoe (from Mapbox) can convert huge amounts of GeoJSON data into MVT
- GDAL with ogr2ogr
- Requires the creation of every tile and delivery upon request of a given z/x/y.
  - 4^z (z=16, 4,294,967,296)
  - Stored file by file (ouch) or PMTiles / archive
- Simple, convenient: throw into cloud storage and deliver directly
- Once made, delivery is *very* fast
- Can take a long time to create and transfer, consume significant amounts of storage, and managing data changes can get tricky

# Vector Tiles: Dynamic

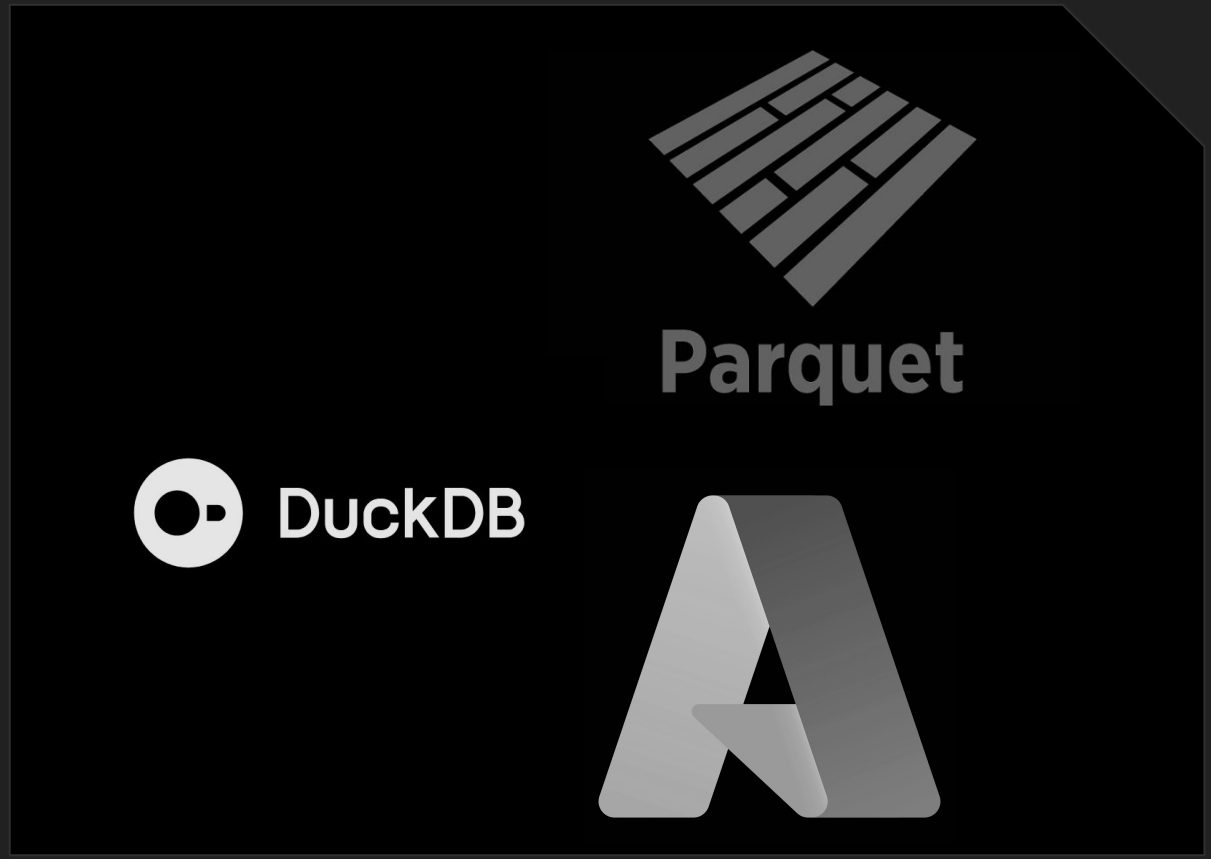
- Examples: Martin, pg_tileserv, tileserver-rs leveraging Postgresql/PostGIS
  - Support for DuckDB as a backend coming online
- Rather than pre-create billions of tiles, generate tiles when they are requested
- Logic: If most tiles will *never* be requested, why spend the compute + storage to have them? What if the data is constantly changing?
- Can be slower, especially with no cache. Consider your use case, user needs, and capabilities.
- However, these options come with the reality that you need to host, configure, and support a database server (PostgreSQL+PostGIS) which introduces costs and scaling constraints

# Vector Tiles: Another option?

No more static tiling and no more database server

# Ingredients

1. (Geo)Parquet
2. DuckDB
3. Azure Storage
4. Azure Functions



# Parquet

- Actual file on disk, transferable, queryable
- Has an actual schema with data types
- Think of it as a souped up CSV
- Columnar storage vs row storage (OLAP vs OLTP)
- Traditional RDBMS (Postgres, MSSQL, etc.) are best at *transactional* workloads
    - As a substitute, we build tons of indexes on columns

# Example

- Consider this tabular data

| ref | speed_tier | description | car | cost | date |
|---|---|---|---|---|---|
| 1 | 10 | slow | pinto | 2000 | 1972 |
| 2 | 10 | slow | tercel | 8000 | 1988 |
| 3 | 20 | fast | m5 | 119500 | 2025 |
| 4 | 20 | fast | lfa | 375000 | 2012 |
| 5 | 20 | fast | r8 | 250000 | 2023 |
| 6 | 20 | fast | corvette | 70000 | 2025 |
| 7 | 50 | lightning | veyron | 1500000 | 2015 |

# Row-based storage

- How similar is data in a row?
- How many indexes do we need?
- If we query for speed_tier=50, we have to know what's in each row (*table scan*)
- Can make an index at the cost of extra space

| ref | speed_tier | description | car | cost | date |
|---|---|---|---|---|---|
| 1 | 10 | slow | pinto | 2000 | 1972 |
| 2 | 10 | slow | tercel | 8000 | 1988 |
| 3 | 20 | fast | m5 | 119500 | 2025 |
| 4 | 20 | fast | lfa | 375000 | 2012 |
| 5 | 20 | fast | r8 | 250000 | 2023 |
| 6 | 20 | fast | corvette | 70000 | 2025 |
| 7 | 50 | lightning | veyron | 1500000 | 2015 |

# Column-based storage

- Instead, store and access data by column. No need to access an entire row.
- The storage is the index - *implicit indexing*
- Helps skip vast amounts of data when querying
- Best for querying lots of records, not looking for a single record (coarse vs fine)

| ref | speed_tier | description | car | cost | date |
|---|---|---|---|---|---|
| 1 | 10 | slow | pinto | 2000 | 1972 |
| 2 | 10 | slow | tercel | 8000 | 1988 |
| 3 | 20 | fast | m5 | 119500 | 2025 |
| 4 | 20 | fast | lfa | 375000 | 2012 |
| 5 | 20 | fast | r8 | 250000 | 2023 |
| 6 | 20 | fast | corvette | 70000 | 2025 |
| 7 | 50 | lightning | veyron | 1500000 | 2015 |

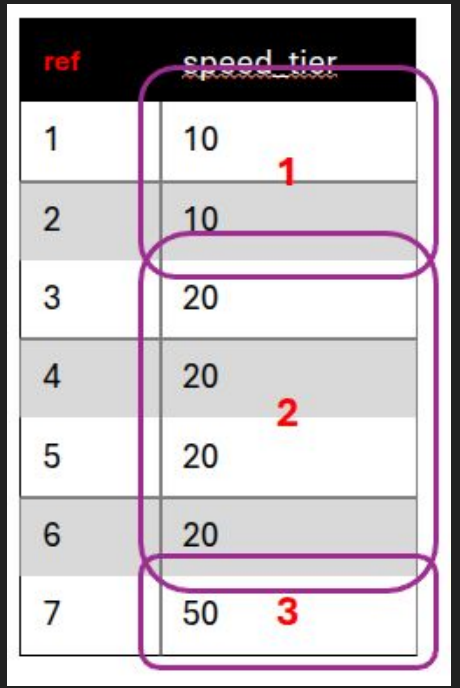
# Column-based storage

- For data with millions of records, row groups help to minimize the amount of data scanned
- Each row group can answer "I have at least one match for where cost<100,000" and if it does not exist it won't be read
- Not useful in our tiny example, but imagine across a larger data set

| rowgrp | ref | speed_tier | description | car | cost | date |
|---|---|---|---|---|---|---|
| 1 | 1 | 10 | slow | pinto | 2000 | 1972 |
| 1 | 2 | 10 | slow | tercel | 8000 | 1988 |
| 1 | 3 | 20 | fast | m5 | 119500 | 2025 |
| 1 | 4 | 20 | fast | lfa | 375000 | 2012 |
| 1 | 5 | 20 | fast | r8 | 250000 | 2023 |
| 2 | 6 | 20 | fast | corvette | 70000 | 2025 |
| 2 | 7 | 50 | lightning | veyron | 1500000 | 2015 |

# Compression

- Data within a column tends to be more similar than data across a row
- Within a column, reduce to unique values. Store just a dictionary and references
  - 1=10
  - 2=20
  - 3=50
- *Warning: simplification!*



# Real-world example

- ~900 million rows of broadband coverage data from the BDC/National Map
- In a database with indexes
  - **350GB**
- In Parquet files (with snappy compression)
  - **14GB**
- ~95% size reduction
  - Sooooo much repetition of data and values!

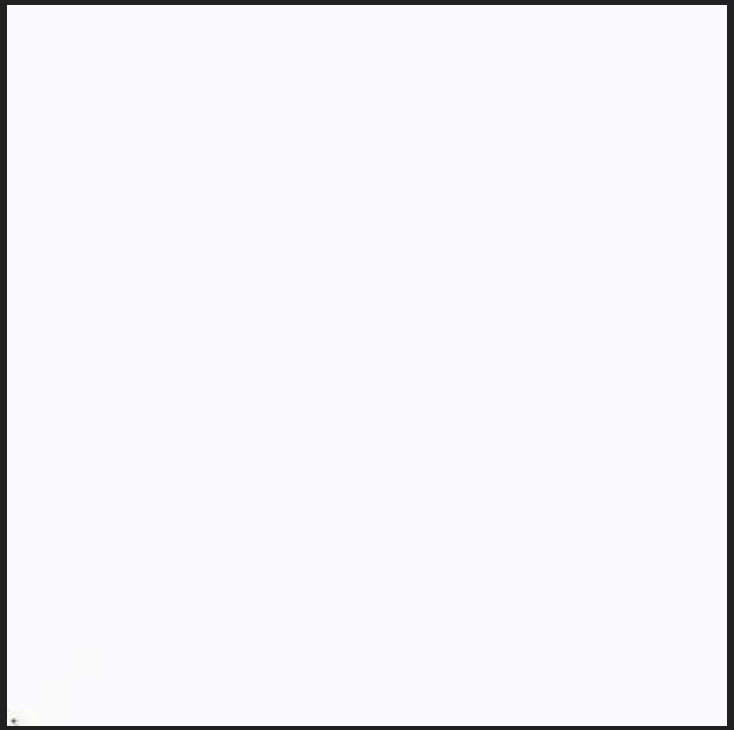
# DuckDB

- In-process analytical database; think SQLite
- Distinction: .duckdb files vs .parquet
- Amazing analytical query engine
- Excellent data workbench for those who are comfortable in SQL
- Read files from local disk or cloud
- Directly query a CSV
- Write out your preferred data format
- Read multiple files using glob syntax
- Multithreading using the vectorized query engine means lots of raw computing power

# Building your GeoParquet files

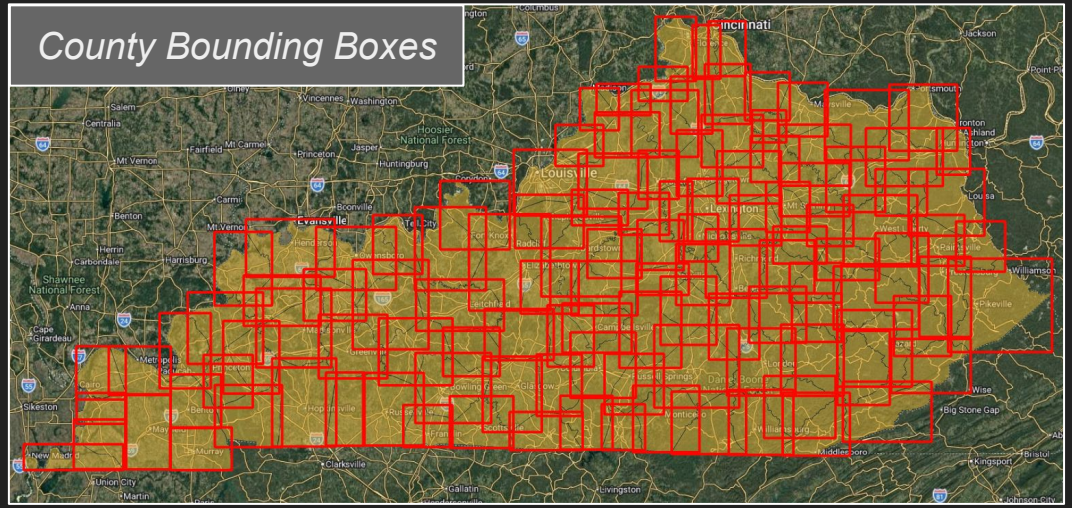
- Keep geographically nearby data grouped together. This applies to both files *and* row groups.
  - We will be using a BBOX-type row filter and want to minimize the number of both files and row groups scanned.
  - Use geographic attribute fields (postal codes, counties, etc.) as well as a geographic ordering approach (ex: Hilbert curve with *order by geom* in PostGIS)
  - https://www.crunchydata.com/blog/waiting-for-postgis-3-hilbert-geometry-sorting
- Keep file size smaller, ex: <100 megabytes.
- Aim for 5-10 row groups per file.
  - We want to be able to prune/filter when querying the data.
- Use Web Mercator (3857) to store geometry to avoid transformation later.
- Use a fast compression method like Snappy to minimize CPU.

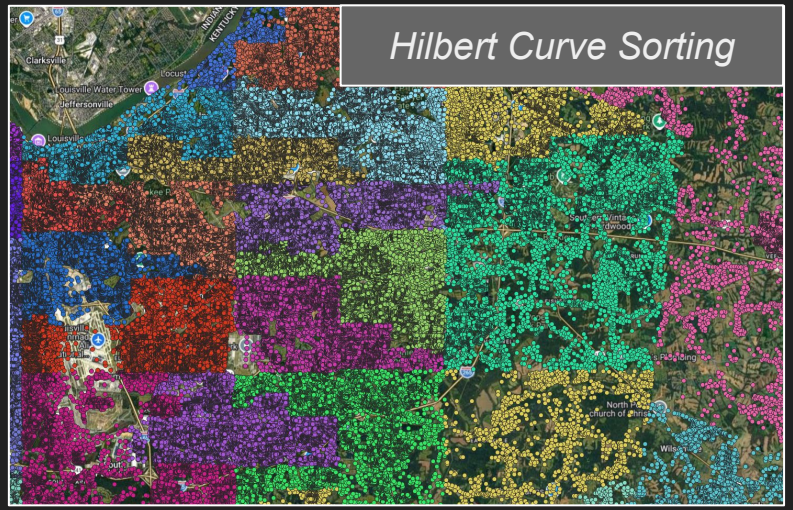
# Hilbert Curve



# File Filtering

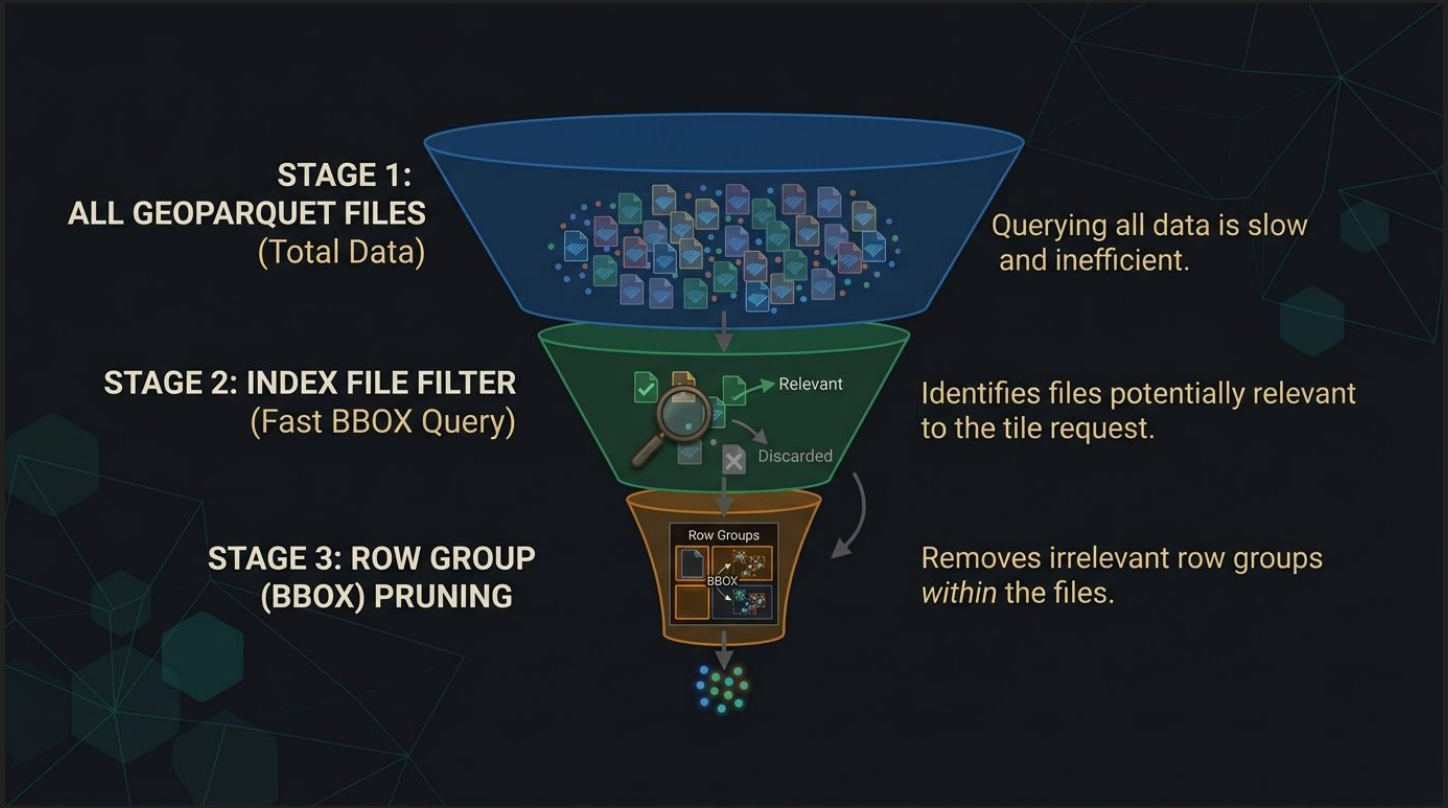
- To minimize files scanned, utilize approaches like hive partitioning or glob matching.
- Run initial query to determine which GeoParquet files to query for a given tile request and then ignore the rest entirely.
- Keep the initial index file small, do not use not true geometries.



** Image generated with AI*

# The Filtering Funnel



# DuckDB

- Extensions
  - azure
  - spatial
- Functions
  - st_tileenvelope()
  - st_asmvtgeom()
  - st_asmvt()
- Azure Specific
  - azure_transport_option_type
  - Azure_read_transfer_concurrency
- Notes for the PostGIS-minded
  - st_transform(geom,**'EPSG:3857'**,**always_xy:=true**)

*Example DuckDB MVT Query*

```
load azure;
load spatial;
set azure_storage_connection_string = '';
set azure_transport_option_type = 'curl';

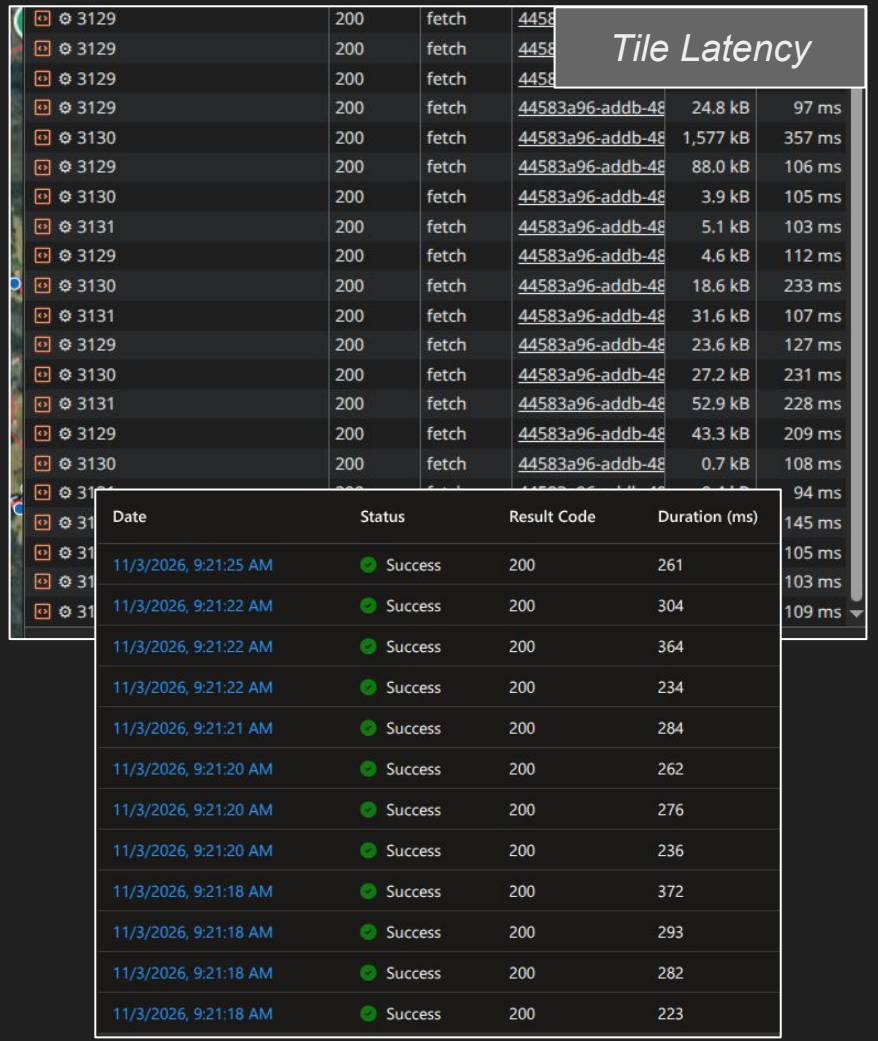
select st_asmvt({
          "attribute1": d.attribute1
         ,"attribute2": d.attribute2
         ,"geometry": st_asmvtgeom(d.geom,st_extent(e.geom))
    },'mvt_layer_name') mvt
from 'az://azduckmvtcontainer/data.parquet' d
join (
    select st_tileenvelope(0,0,0) geom
) e on st_intersects(e.geom,d.geom)=true;
```

# Azure Storage, Functions, and More

- Mandatory
  - **Bundle DuckDB extensions with your function.**
    - Must avoid fresh installs when new instances are created.
  - Same region for cloud resources (storage, function, compute).
  - Keep dependencies minimal.
- More Performance / Other Ideas
  - Use Premium_LRS or Data Lake Gen2 storage for lower latencies.
  - Use a premium plan or utilize pre-warmed function instances to avoid cold starts.
  - Utilize a cache
    - Different set of implementation decisions (TTL, storage medium, delivery method)
  - Simplify, sample, or thin geometries at lower zoom levels.
    - Collapse points so tile sizes remain small, ex: **group by x // 100,y // 100** (your data is in meters right?)
    - Simplify polygons/lines, ex: **st_simplify(geom,100)**
  - Return fewer attributes (smaller tile sizes)
  - Hybridized delivery: pre-compute static tiles at lower zoom levels (ex: 0-10, dynamic at 11+)

# Code and Demo

- DuckDB for ETL
  - **./data/script.sql**
- Azure Resources (Storage, Function)
  - **./function/script.sh**
  - **./function/function_app.py**
- Demo Site
  - **./web/***



# On The Web / The End

- Presentation Repository
  - https://github.com/jesseamundsen/azduckmvt
- Contact
  - jesse.amundsen@gmail.com
  - https://github.com/jesseamundsen
  - https://www.linkedin.com/in/jesseamundsen
  - https://jesseamundsen.github.io
- Q&A Time

*Repository / Links*

